

**D.Y. Patil Academic Education Excellence Federation's****Dr. D. Y. Patil Technical Campus****Polytechnic | Engineering | MCA**

(Approved by AICTE, Recognized by DTE, Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University, Pune)
Sr.No.32/1/A/7, Near Talegaon Railway Station, A/P Varale, Talegaon Dabhade, Tal-Maval, Dist-Pune 410 507

Website: www.dypatiltes.com Email: principal@dypatiltes.com

AICTE ID: 1- 43358498993 | SPPU PUN CODE: CMMP021930 | ISHE CODE: C -71504 | DTE CODE: 6991

Dr. D. Y. Patil

Founder Dr. D. Y. Patil Group

Dr. Sushant V. Patil

Chairman

Adv. Anuja Sushant Patil

Director

Dr. Suresh Shirbahadurkar

Principal

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

TOPIC: Java Programing Language**Prepared By: Avinash Sunklekar**

JAVA PROGRAMMING LANGUAGE

Java is a popular, high-level programming language used to develop different types of applications. It was developed by Sun Microsystems and released in 1995. Java is simple, secure, and easy to learn. It follows the Object-Oriented Programming (OOP) concept. Java supports the principle of “Write Once, Run Anywhere,” which means a Java program can run on different operating systems. Java code is converted into bytecode by the Java compiler, and this bytecode runs on the Java Virtual Machine (JVM).

Java is used to develop web applications, mobile applications, desktop software, and enterprise applications. It provides important features such as inheritance, polymorphism, encapsulation, and abstraction. Java also supports exception handling and multithreading. Due to its powerful features, portability, and security, Java is widely used in software development.

Basic Syntactical Constructs in Java

1.1 Fundamentals of Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data in the form of fields (often known as attributes or properties) and code in the form of procedures (often known as methods). OOP reorganizes program structure around data rather than procedural logic.

Core Features of OOP Architecture

- **Emphasis on Data:** Data is treated as a critical element in the program development and is restricted from moving freely around the system.
- **Program Decomposition:** Programs are decomposed into entities called *Objects*, which bundle data and related functions.
- **Data Hiding & Security:** Data is hidden and cannot be accessed by external functions, providing higher security compared to Procedure-Oriented Programming (POP).
- **Inter-Object Communication:** Objects communicate with each other through well-defined interface methods.
- **Extensibility:** New data items and methods can be added easily whenever required without modifying existing modules.
- **Bottom-Up Approach:** OOP follows a bottom-up approach in program design, building complex software from foundational object primitives.

1.2 Basic Concepts of OOP & Benefits/Applications

Basic Concepts

1. **Objects:** Basic runtime entities in an object-oriented system. An object represents a real-world entity (e.g., student, bank account, table) and encapsulates both state (data) and behavior (methods).
2. **Classes:** User-defined data types that serve as blueprints for objects. A class encapsulates data members and member functions. Defining a class does not allocate memory for its members until an object is instantiated.
3. **Data Abstraction:** The act of representing essential features without including background details or explanations. It reduces complexity and allows focusing on core interactions.
4. **Data Encapsulation:** The wrapping up of data and functions into a single unit (class). It prevents direct access to data from outside the class (data hiding).
5. **Inheritance:** The process by which an object of one class acquires the properties and behaviors of an object of another class. It promotes code reusability and hierarchical classification.

6. **Polymorphism:** Derived from Greek (Poly = many, Morphism = forms), it refers to the capability of a method or operator to take more than one form. Java supports compile-time polymorphism (Method Overloading) and runtime polymorphism (Method Overriding / Virtual Functions).
7. **Dynamic Binding:** Linking of a function call to the code to be executed at runtime rather than compile time. Associated with polymorphism and inheritance.
8. **Message Passing:** Objects communicate by sending and receiving messages (invoking methods on other objects).

Benefits and Real-World Applications

Benefits: Enhanced code reusability through inheritance, seamless software maintenance, effective software complexity management, easy scalability from small to enterprise applications, and work partitioning across development teams.

Applications: Real-time software systems, simulation and modeling, Object-Oriented Databases (OODBMS), Artificial Intelligence and Expert Systems, Computer-Aided Design (CAD/CAM), Hypertext/Hypermedia systems, and parallel processing networks.

1.3 Evolution and History of Java

Java was developed in 1991 by **James Gosling** and his team (The Green Team) at **Sun Microsystems**, USA. Initially designed for small, consumer electronic devices like set-top boxes, the project was originally named **Oak** after an oak tree standing outside Gosling's office.

In 1995, Oak was renamed **Java** (inspired by Java coffee) and re-architected to focus on the emerging World Wide Web. Java's design philosophy prioritized portable, secure, and robust execution across heterogeneous computer networks, leading to the motto: *"Write Once, Run Anywhere" (WORA)*.

1.4 Detailed Features of Java Architecture

Feature	Detailed Architectural Description
Compiled & Interpreted	Java uses a two-stage execution process. Source code (<code>.java</code>) is compiled by <code>javac</code> into intermediate, platform-neutral Bytecode (<code>.class</code>). The Java Virtual Machine (JVM) interpreter translates Bytecode into native machine instructions at execution time.
Platform Independent & Portable	Bytecode is independent of operating system architecture. Java applications can run unchanged on Windows, Linux, macOS, or Unix systems equipped with a compatible JVM.
True Object-Oriented	Everything in Java (except primitive data types) is encapsulated within classes and objects. Java features a vast standard class library (JSL) structured into packages.
Robust & Secure	Strict compile-time and runtime type checking, automatic memory management (Garbage Collection), built-in exception handling, elimination of explicit pointers, and bytecode verification ensure memory protection and execution safety.
Distributed	Java includes extensive networking capabilities (RMI, Sockets, HTTP handling) facilitating distributed computing and remote object access across local networks and the Internet.
Multi-threaded & Interactive	Native support for multithreaded programming allows concurrent execution of multiple execution threads within a single process, maximizing CPU utilization and interactive responsiveness.
High Performance	While interpreted, performance is heavily optimized using Just-In-Time (JIT) compilers that translate frequently executed bytecode sequences into native machine code at runtime.
Dynamic & Extensible	Dynamic loading of classes on-demand allows code updates without restarting applications. Supports Native Methods (C/C++ integration via JNI).

1.5 Architectural Differences: Java vs C vs C++

Java vs C

- Java is an Object-Oriented programming language; C is a Procedure-Oriented language.
- Java does not support keywords like `sizeof` or `typedef` .
- Java eliminates complex data structures like `struct` and `union` .
- Java does not feature memory storage class modifiers: `auto` , `register` , `extern` , `signed` , or `unsigned` .
- Java eliminates explicit pointer arithmetic and pointer references to avoid direct memory manipulation.
- Java does not use preprocessor directives (no `#define` or `#include`).
- Java introduces labeled `break` and `continue` statements and operators like `instanceof` .

Java vs C++

- C++ is a hybrid (multi-paradigm) language; Java is a true object-oriented language.
- Java does not support operator overloading or global variables.
- Java does not support explicit pointers or explicit memory deallocation (no `destructors` ; uses `finalize()` / automatic garbage collection).
- Java does not support multiple inheritance of classes directly; it uses **Interfaces** instead.
- Java replaces C++ header files with structured **Packages**.

1.6 Java Development Environment (JDK, JRE, JVM, JSL)

The Java development ecosystem consists of several interconnected software layers:

- **JDK (Java Development Kit):** The full development toolset containing compiler (`javac`), interpreter (`java`), debugger (`jdb`), documentation tool (`javadoc`), archiver (`jar`), and disassembler (`javap`).
- **JRE (Java Runtime Environment):** The execution environment that contains the JVM, core class libraries, and supporting runtime binaries required to execute Java programs.
- **JVM (Java Virtual Machine):** The abstract virtual machine engine that executes bytecode, manages memory, performs garbage collection, and provides operating system abstraction.
- **JSL (Java Standard Library / API):** Pre-compiled utility classes categorized into standard packages (e.g., `java.lang` , `java.util` , `java.io`).

1.7 Java Comments & Command Line Arguments

Comments are ignored by the Java compiler and serve as inline documentation:

```
// Single-line comment: Used for short explanations.

/*
  Multi-line comment: Used for extended multi-line
  descriptions and code documentation.
*/
```

Command Line Arguments

Command line arguments allow passing parameters to the `main()` method at execution time. Arguments are passed as an array of `String` objects to `String args[]` .

```
// File: CommandLineDemo.java
class CommandLineDemo {
    public static void main(String args[]) {
        if (args.length < 2) {
            System.out.println("Please provide two integer arguments.");
            return;
        }
        int num1 = Integer.parseInt(args[0]);
        int num2 = Integer.parseInt(args[1]);
        int sum = num1 + num2;
        System.out.println("Sum of inputs = " + sum);
    }
}
// Execution: java CommandLineDemo 100 200
// Output: Sum of inputs = 300
```

1.8 User Input Processing using Scanner Class

The `Scanner` class (found in `java.util`) provides utility methods to parse primitive data types and strings from standard input (`System.in`).

Scanner Method	Input Description
<code>nextInt()</code>	Reads an <code>int</code> integer value from the stream.
<code>nextFloat()</code>	Reads a <code>float</code> single-precision floating value.
<code>nextDouble()</code>	Reads a <code>double</code> double-precision floating value.
<code>nextLong()</code>	Reads a <code>long</code> 64-bit integer value.
<code>nextBoolean()</code>	Reads a <code>boolean</code> (true/false) value.
<code>nextLine()</code>	Reads an entire line of text until a newline character.
<code>next()</code>	Reads a single whitespace-delimited word.

```
import java.util.Scanner;

class ScannerDemo {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Roll Number: ");
        int roll = sc.nextInt();
        System.out.print("Enter Full Name: ");
        sc.nextLine(); // Consume newline
        String name = sc.nextLine();

        System.out.println("Student: " + name + " | Roll No: " + roll);
    }
}
```

1.9 Mathematical Operations with java.lang.Math

The `Math` class in `java.lang` provides static methods for trigonometric, logarithmic, and mathematical functions.

```
class MathDemo {
    public static void main(String args[]) {
        double a = 12.0, b = 15.0;
        System.out.println("Max value: " + Math.max(a, b));
        System.out.println("Min value: " + Math.min(a, b));
        System.out.println("Square root of 81: " + Math.sqrt(81));
        System.out.println("Power (2^4): " + Math.pow(2, 4));
        System.out.println("Absolute value (-45): " + Math.abs(-45));
        System.out.println("Rounded value (200.67): " + Math.round(200.67));
    }
}
```

1.10 Data Types, Variables, and Memory Allocation

Java supports primitive data types and reference data types.

Type Category	Data Type	Storage Size	Default Value	Valid Range / Description
Integer	<code>byte</code>	1 byte (8 bits)	<code>0</code>	-128 to 127
	<code>short</code>	2 bytes (16 bits)	<code>0</code>	-32,768 to 32,767
	<code>int</code>	4 bytes (32 bits)	<code>0</code>	-2^{31} to $2^{31}-1$
	<code>long</code>	8 bytes (64 bits)	<code>0L</code>	-2^{63} to $2^{63}-1$
Floating	<code>float</code>	4 bytes (32 bits)	<code>0.0f</code>	Single precision IEEE 754
	<code>double</code>	8 bytes (64 bits)	<code>0.0d</code>	Double precision IEEE 754
Character	<code>char</code>	2 bytes (16 bits)	<code>'</code>	
Boolean	<code>boolean</code>	1 bit (logical)	<code>false</code>	<code>true</code> or <code>false</code>

Scope of Variables

- **Instance Variables:** Declared inside a class but outside methods. Allocated when objects are created; separate memory copy per object instance.
- **Class (Static) Variables:** Declared with `static` keyword inside a class. Single shared copy across all class instances.
- **Local Variables:** Declared inside a block, method, or constructor. Created upon method entry and destroyed upon exit. Must be explicitly initialized before use.

1.11 Type Casting: Implicit vs Explicit Conversion

Type casting is the process of converting a variable from one data type to another.

- **Implicit Type Conversion (Widening):** Handled automatically by the compiler when converting a smaller data type to a larger compatible type without risk of data loss.
- **Explicit Type Conversion (Narrowing):** Performed manually by the programmer using target type syntax when assigning a larger type to a smaller type. May result in precision loss.

```

class TypeCastingDemo {
    public static void main(String args[]) {
        // Implicit Casting (int -> float)
        int numInt = 70;
        float numFloat = numInt; // Automatic widening
        System.out.println("Implicit Cast Value: " + numFloat);

        // Explicit Casting (double -> int)
        double price = 199.99;
        int castedPrice = (int) price; // Explicit narrowing (fractional part truncated)
        System.out.println("Explicit Cast Value: " + castedPrice);
    }
}

```

1.12 Operators, Expressions, and instanceof Operator

Java provides comprehensive operators categorized into Arithmetic (`+, -, *, /, %`), Relational (`<, >, <=, >=, ==, !=`), Logical (`&&, ||, !`), Bitwise (`&, |, ^, <<, >>, ~`), Assignment (`=, +=, -=`), and Conditional (`?:`).

The `instanceof` Operator

The `instanceof` operator checks whether an object is an instance of a specific class, subclass, or interface at runtime. Returns a boolean value.

```

import java.util.Scanner;

class InstanceOfDemo {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        if (sc instanceof Scanner) {
            System.out.println("sc is an active instance of java.util.Scanner");
        }
    }
}

```

1.13 Decision Making Control Structures

Control statements govern execution flow based on conditional evaluations.

```
// 1. simple if & if-else
if (number % 2 == 0) {
    System.out.println("Even Number");
} else {
    System.out.println("Odd Number");
}

// 2. else-if ladder
if (marks >= 75) {
    System.out.println("Distinction");
} else if (marks >= 60) {
    System.out.println("First Class");
} else if (marks >= 40) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}

// 3. switch-case statement
switch (choice) {
    case 1:
        System.out.println("Option 1 Selected");
        break;
    case 2:
        System.out.println("Option 2 Selected");
        break;
    default:
        System.out.println("Invalid Selection");
}
```

1.14 Looping Constructs (for, while, do-while, enhanced for)

```
// Standard for loop
for (int i = 1; i <= 5; i++) {
    System.out.println("Iteration: " + i);
}

// Standard while loop
int w = 1;
while (w <= 5) {
    System.out.println("While Count: " + w);
    w++;
}

// Standard do-while loop (Executes at least once)
int d = 1;
do {
    System.out.println("Do-While Count: " + d);
    d++;
} while (d <= 5);

// Enhanced for loop (For-Each loop over arrays)
int numbers[] = {10, 20, 30, 40, 50};
for (int val : numbers) {
    System.out.println("Array Value: " + val);
}
```

Classes, Objects, Methods, Vectors, and Strings

2.1 Defining Classes and Instantiating Objects

A class acts as a template defining state (instance variables) and behavior (methods). Objects are specific instances created from this template.

```
class Addition {
    // Instance Variables
    int num1, num2, result;

    // Member Method to set data
    void setData(int a, int b) {
        num1 = a;
        num2 = b;
    }

    // Member Method to display result
    void calculateAndDisplay() {
        result = num1 + num2;
        System.out.println("Addition Result = " + result);
    }
}

class ClassDemo {
    public static void main(String args[]) {
        // Object creation using 'new' operator
        Addition obj = new Addition();
        obj.setData(100, 50);
        obj.calculateAndDisplay();
    }
}
```

2.2 Accessing Class Members and Memory Layout

Class members are accessed using object instance variables and the dot (`.`) operator: `objectName.variableName` or `objectName.methodName()`.

Memory Allocation: When a class variable is declared (e.g., `Addition obj;`), reference memory is allocated on the Stack. Executing `new Addition()` allocates actual instance memory on the Heap and returns the memory address reference to `obj`.

2.3 Array of Objects Concept and Memory Structuring

An array of objects stores reference variables pointing to individual class instances.

```
class Student {
    int id;
    String name;

    void setDetails(int i, String n) {
        id = i;
        name = n;
    }

    void display() {
        System.out.println("ID: " + id + " | Name: " + name);
    }
}

class ObjectArrayDemo {
    public static void main(String args[]) {
        // Allocate array storing 3 references
        Student list[] = new Student[3];

        // Instantiate individual objects
        for (int i = 0; i < list.length; i++) {
            list[i] = new Student();
        }

        list[0].setDetails(101, "Alice");
        list[1].setDetails(102, "Bob");
        list[2].setDetails(103, "Charlie");

        for (Student s : list) {
            s.display();
        }
    }
}
```

2.4 Constructors: Default, Parameterized, and Copy Constructors

A constructor is a special block of code that initializes a newly created object. It shares the exact name of the class and has no explicit return type.

```

class Item {
    int code;

    // 1. Default Constructor
    Item() {
        code = 100;
        System.out.println("Default Constructor Executed");
    }

    // 2. Parameterized Constructor
    Item(int c) {
        code = c;
        System.out.println("Parameterized Constructor Executed");
    }

    // 3. Copy Constructor (Passes object reference)
    Item(Item obj) {
        this.code = obj.code;
        System.out.println("Copy Constructor Executed");
    }

    void display() {
        System.out.println("Item Code: " + code);
    }
}

class ConstructorDemo {
    public static void main(String args[]) {
        Item item1 = new Item();           // Calls Default
        Item item2 = new Item(500);        // Calls Parameterized
        Item item3 = new Item(item2);      // Calls Copy

        item1.display();
        item2.display();
        item3.display();
    }
}

```

2.5 Constructor Overloading Concepts and Applications

Constructor overloading occurs when a class contains multiple constructors with identical names but differing parameter lists (different number, types, or order of parameters).

```

class Room {
    float length, width;

    // Constructor with 1 argument (Square room)
    Room(float side) {
        length = side;
        width = side;
    }

    // Constructor with 2 arguments (Rectangular room)
    Room(float l, float w) {
        length = l;
        width = w;
    }

    void area() {
        System.out.println("Room Area = " + (length * width));
    }
}

class OverloadDemo {
    public static void main(String args[]) {
        Room squareRoom = new Room(10.5f);
        Room rectRoom = new Room(12.0f, 15.0f);

        squareRoom.area();
        rectRoom.area();
    }
}

```

2.6 Static Members: Data Members and Static Methods

Static members belong to the class itself rather than to individual instances. Only a single copy of a static member exists in memory, shared across all objects of that class.

Static Member Rules

- Static variables are initialized when the class is loaded into memory.
- Static methods can only directly access static variables and execute static methods within the class.
- Static methods cannot utilize `this` or `super` keywords.
- Static members are accessed via `ClassName.memberName`.

```

class StaticDemo {
    int id; // Instance variable
    static int count = 0; // Class variable

    StaticDemo(int id) {
        this.id = id;
        count++; // Increments shared static counter
    }

    static void displayCount() {
        System.out.println("Total Objects Created: " + count);
    }
}

class StaticTest {
    public static void main(String args[]) {
        StaticDemo o1 = new StaticDemo(101);
        StaticDemo o2 = new StaticDemo(102);
        StaticDemo o3 = new StaticDemo(103);

        StaticDemo.displayCount(); // Output: 3
    }
}

```

2.7 Access Control Specifiers / Visibility Control

Access Specifier	Same Class	Subclass (Same Pkg)	Non-Subclass (Same Pkg)	Subclass (Diff Pkg)	World (Diff Pkg)
<code>private</code>	Yes	No	No	No	No
<i>Friendly (Default)</i>	Yes	Yes	Yes	No	No
<code>protected</code>	Yes	Yes	Yes	Yes	No
<code>public</code>	Yes	Yes	Yes	Yes	Yes

2.8 One-Dimensional and Two-Dimensional Arrays

```
// 1D Array Processing
int singleArr[] = {10, 20, 30, 40, 50};
for (int i = 0; i < singleArr.length; i++) {
    System.out.print(singleArr[i] + " ");
}

// 2D Matrix Array Processing
int matrix[][] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
for (int row = 0; row < 3; row++) {
    for (int col = 0; col < 3; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println();
}
```

2.9 Dynamic Collections: Vectors and Vector Operations

The `Vector` class (part of `java.util`) implements a dynamic expandable array of objects. Unlike standard arrays, vectors can shrink or grow automatically as items are added or removed.

Vector Method	Functional Description
<code>addElement(Object item)</code>	Appends specified element to the end of vector.
<code>elementAt(int index)</code>	Returns object reference stored at specified index.
<code>size()</code>	Returns total count of elements currently stored.
<code>removeElement(Object item)</code>	Removes first matching occurrence of element.
<code>removeElementAt(int index)</code>	Deletes element located at specified index.
<code>insertElementAt(Object item, int index)</code>	Inserts element into specified index position.
<code>removeAllElements()</code>	Clears all stored elements from vector.

```
import java.util.Vector;

class VectorDemo {
    public static void main(String args[]) {
        Vector vec = new Vector();
        vec.addElement(Integer.valueOf(10));
        vec.addElement(Integer.valueOf(30));
        vec.addElement(Integer.valueOf(60));

        System.out.println("Vector Size: " + vec.size());
        vec.insertElementAt(Integer.valueOf(20), 1); // Insert 20 at index 1
        vec.removeElementAt(3); // Remove 60

        for (int i = 0; i < vec.size(); i++) {
            System.out.println("Element " + i + ": " + vec.elementAt(i));
        }
    }
}
```

2.10 Primitive Type Wrappers: Wrapper Classes

Wrapper classes encapsulate primitive data types into Java objects, allowing primitives to participate in object-centric collection APIs like `Vector`.

Primitive Type	Wrapper Class	Constructor Conversion	Object to Primitive Method
<code>byte</code>	<code>Byte</code>	<code>new Byte(b)</code>	<code>byteValue()</code>
<code>short</code>	<code>Short</code>	<code>new Short(s)</code>	<code>shortValue()</code>
<code>int</code>	<code>Integer</code>	<code>new Integer(i)</code>	<code>intValue()</code>
<code>long</code>	<code>Long</code>	<code>new Long(l)</code>	<code>longValue()</code>
<code>float</code>	<code>Float</code>	<code>new Float(f)</code>	<code>floatValue()</code>
<code>double</code>	<code>Double</code>	<code>new Double(d)</code>	<code>doubleValue()</code>
<code>char</code>	<code>Character</code>	<code>new Character(c)</code>	<code>charValue()</code>
<code>boolean</code>	<code>Boolean</code>	<code>new Boolean(b)</code>	<code>booleanValue()</code>

```

class WrapperDemo {
    public static void main(String args[]) {
        int val = 100;
        Integer obj = Integer.valueOf(val); // Primitive to Object (Boxing)
        int primitiveVal = obj.intValue(); // Object to Primitive (Unboxing)

        String numStr = "500";
        int parsedNum = Integer.parseInt(numStr); // String parsing to primitive
        System.out.println("Parsed Numeric Value: " + parsedNum);
    }
}

```

2.11 Immutable Character Sequences: String Class

In Java, `String` objects are immutable sequence values stored inside memory pool storage. Modifying a string creates a new string instance.

```

class StringMethodsDemo {
    public static void main(String args[]) {
        String str = "Java Programming";

        System.out.println("Length: " + str.length());
        System.out.println("Char at index 2: " + str.charAt(2));
        System.out.println("Lowercase: " + str.toLowerCase());
        System.out.println("Uppercase: " + str.toUpperCase());
        System.out.println("Substring (0, 4): " + str.substring(0, 4));
        System.out.println("Equals check: " + str.equals("Java Programming"));
        System.out.println("Replaced: " + str.replace('a', 'x'));
    }
}

```

2.12 Mutable Sequences: StringBuffer Class

The `StringBuffer` class creates dynamic, mutable character strings that can grow or shrink without spawning new intermediate memory instances.

```

class StringBufferDemo {
    public static void main(String args[]) {
        StringBuffer sb = new StringBuffer("Java");
        sb.append(" Language");
        System.out.println("Appended String: " + sb);

        sb.insert(4, " Core");
        System.out.println("Inserted String: " + sb);

        sb.setCharAt(0, 'j');
        System.out.println("Modified Char: " + sb);
    }
}

```

2.13 Technical Comparison Tables

Array vs Vector

Feature	Array	Vector
Sizing	Fixed length set during initialization.	Dynamic expandable size.
Class Type	Language data structure construct.	Standard library collection class (<code>java.util</code>).
Data Storage	Stores primitives & object references.	Stores reference objects only.
Synchronization	Not thread-synchronized.	Synchronized thread-safe methods.
Modification	Direct indexing access.	Dedicated methods (<code>addElement</code> , <code>removeElement</code>).

String vs StringBuffer

Feature	String	StringBuffer
Mutability	Immutable (Cannot be modified after creation).	Mutable (Can grow or modify contents directly).
Performance	Slower for frequent concatenation.	High performance for heavy modifications.
Memory Storage	Stored in String Constant Pool or Heap.	Stored directly in Heap memory.
Equality	Overrides <code>equals()</code> for value check.	Does not override <code>equals()</code> (uses reference check).

Inheritance, Interfaces, and Packages

3.1 Principles of Inheritance & Derived Classes

Inheritance is the mechanism of deriving a new class (subclass / derived class) from an existing class (superclass / base class). Subclasses inherit fields and methods from superclasses using the `extends` keyword, maximizing software reusability.

3.2 Types of Inheritance

- **Single Inheritance:** A subclass inherits directly from one base class.
- **Multilevel Inheritance:** A class inherits from a subclass, forming an inheritance chain.
- **Hierarchical Inheritance:** Multiple subclasses inherit from a single base class.
- **Hybrid Inheritance:** Combination of two or more inheritance types.
- **Multiple Inheritance:** Inheriting from multiple base classes simultaneously. *(Not supported directly for classes in Java; achieved using Interfaces).*

```
// Single Inheritance Architecture
class Student {
    int rollNo;
    String name;

    void setStudent(int r, String n) {
        rollNo = r;
        name = n;
    }
}

class Test extends Student {
    int marks1, marks2;

    void setMarks(int m1, int m2) {
        marks1 = m1;
        marks2 = m2;
    }

    void displayResult() {
        System.out.println("Roll: " + rollNo + " | Name: " + name);
        System.out.println("Total Marks: " + (marks1 + marks2));
    }
}
```

3.3 Method Overriding & Method Shadowing Rules

Method overriding occurs when a subclass defines a method with the exact same name, return type, and signature as a method in its superclass. The subclass method overrides the superclass implementation at runtime.

```
class Base {
    void show() {
        System.out.println("Base class display method");
    }
}

class Derived extends Base {
    @Override
    void show() {
        super.show(); // Calls hidden base class method
        System.out.println("Derived class display method");
    }
}
```

3.4 Constructor Invocation in Inheritance Hierarchy (super keyword)

Superclass constructors are not inherited by subclasses. A subclass constructor invokes its superclass constructor using `super()` or `super(argument_list)`. The `super()` call must be the **very first statement** inside the subclass constructor.

```
class Parent {
    int x;

    Parent(int value) {
        x = value;
        System.out.println("Parent Constructor initialized: x = " + x);
    }
}

class Child extends Parent {
    int y;

    Child(int val1, int val2) {
        super(val1); // Explicitly invoke superclass parameterized constructor
        y = val2;
        System.out.println("Child Constructor initialized: y = " + y);
    }
}
```

3.5 Deep Dive into the final Keyword (Variables, Methods, Classes)

Application	Syntax Pattern	Behavioral Impact
Final Variable	<code>final float PI = 3.14159f;</code>	Converts variable into an immutable constant. Reassignment raises a compilation error.
Final Method	<code>final void compute() { ... }</code>	Prevents method from being overridden by derived subclasses.
Final Class	<code>final class Base { ... }</code>	Prevents class from being inherited. Subclassing raises a compilation error.

3.6 Interfaces: Abstract Specifications and Implementations

An **Interface** is a syntactical contract specifying abstract methods and static final constants. Interfaces achieve complete abstraction and enable multiple inheritance in Java. Classes fulfill interface contracts using the `implements` keyword.

```
interface Area {
    float PI = 3.14f; // Implicitly public static final
    void compute(int radius); // Implicitly public abstract
}

class Circle implements Area {
    public void compute(int radius) {
        float result = PI * radius * radius;
        System.out.println("Calculated Circle Area = " + result);
    }

    public static void main(String args[]) {
        Circle c = new Circle();
        c.compute(10);
    }
}
```

3.7 Inter-Class & Inter-Interface Relationships

- Class **extends** Class
- Class **implements** Interface
- Interface **extends** Interface
- *(Banned: Interface cannot implement or extend a Class)*

```
// Multiple Interface Implementation Architecture
interface Abc {
    void display();
}

interface Xyz {
    void show();
}

class ImplementationNode implements Abc, Xyz {
    public void display() {
        System.out.println("Abc interface method implemented");
    }

    public void show() {
        System.out.println("Xyz interface method implemented");
    }
}
```

3.8 Technical Comparison Tables

Abstract Class vs Interface

Feature	Abstract Class	Interface
Method Types	Abstract and concrete (implemented) methods.	Abstract methods only (prior to Java 8).
Inheritance	Single inheritance (extends one class).	Multiple inheritance (implements multiple interfaces).
Variables	Final, non-final, static, non-static.	Static and final variables only.
Abstraction	Partial abstraction (0 to 100%).	Full abstraction (100%).

Class vs Interface

Feature	Class	Interface
Instantiation	Objects can be directly instantiated.	Cannot be instantiated directly.
Constructors	Contains constructors for initialization.	No constructors.
Access Modifiers	Members can be private, protected, public.	Members are implicitly public.
Keyword	Declared using <code>class</code> .	Declared using <code>interface</code> .

3.9 Package Encapsulation & Standard Java API Packages

A **Package** is a namespace container grouping related classes and interfaces into directory structures, avoiding naming collisions and managing access permissions.

Built-In Package	Functionality & Contents
<code>java.lang</code>	Core language classes (primitive types, Math, String, Thread, Exception). Automatically imported.
<code>java.util</code>	Utility collections, vector structures, scanner, date/time functions.
<code>java.io</code>	File reading, writing, and stream I/O operations.
<code>java.net</code>	Network communication, socket handling, URL connections.
<code>java.awt</code>	Abstract Window Toolkit for Graphical User Interfaces (GUI).
<code>java.applet</code>	Classes for embedding Web applets.

3.10 User-Defined Packages: Creation and Maintenance

1. Declare the package at the top of the Java source file using `package packageName;`.
2. Declare classes within the package as `public` to allow external access.
3. Save files in directory structures matching package names and compile using `javac -d . FileName.java`.

```
// File: MathOperations/Addition.java
package MathOperations;

public class Addition {
    public void add(int a, int b) {
        System.out.println("Sum = " + (a + b));
    }
}
```

3.11 Package Import Strategies and Static Imports

- `import packageName.*;` - Imports all public classes within the package.
- `import packageName.ClassName;` - Imports only the specified class.
- **Fully Qualified Name:** Referencing classes using full path without import (e.g., `java.util.Date d = new java.util.Date();`).
- **Static Import:** Direct import of static members without referencing class names (e.g., `import static java.lang.System.*;` allows using `out.println()` directly).

Multithreaded Programming and Exception Handling

4.1 Introduction to Multithreading & Multitasking

Multithreading is a feature that enables concurrent execution of two or more threads within a single program. A **Thread** is a lightweight execution subprocess sharing a common memory area.

4.2 Multithreading vs Multiprocessing Architecture

Architectural Metric	Multithreading	Multiprocessing
Execution Unit	Lightweight execution thread.	Heavyweight system process.
Memory Allocation	Threads share common process memory space.	Each process gets separate dedicated memory.
Context Switching Cost	Low context switching overhead.	High context switching overhead.
Resource Efficiency	Saves memory and processor time.	Higher system resource footprint.

4.3 Complete Life Cycle of a Thread

A Java thread passes through discrete states managed by the JVM:

1. **Newborn State:** Thread instance created via `new` operator, prior to calling `start()`.
2. **Runnable State:** Thread is ready for execution after `start()` is called and is waiting in the CPU scheduler queue.
3. **Running State:** CPU allocates time slices to execute the thread's `run()` method.
4. **Blocked / Waiting State:** Thread is temporarily suspended via `sleep()`, `wait()`, or I/O blockage. Returns to Runnable via `resume()`, `notify()`, or time expiration.
5. **Dead State:** Thread terminates when execution of `run()` completes or `stop()` is invoked.

4.4 Thread Creation Methods: Extending Thread vs Implementing Runnable

Method 1: Extending the Thread Class

```
class WorkerThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Worker Thread Executing: Count " + i);
        }
    }
}

class ThreadExecution {
    public static void main(String args[]) {
        WorkerThread t1 = new WorkerThread();
        t1.start(); // Spawns thread and invokes run()
    }
}
```

Method 2: Implementing the Runnable Interface

```
class RunnableTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable Task Execution: Count " + i);
        }
    }
}

class RunnableExecution {
    public static void main(String args[]) {
        RunnableTask task = new RunnableTask();
        Thread threadInstance = new Thread(task);
        threadInstance.start();
    }
}
```

4.5 Thread Priorities & Preemptive Scheduling

Java assigns integer priorities (ranging from 1 to 10) to threads to manage CPU scheduling:

- `Thread.MIN_PRIORITY` (Value: 1)
- `Thread.NORM_PRIORITY` (Value: 5 - Default)
- `Thread.MAX_PRIORITY` (Value: 10)

```
class PriorityDemo extends Thread {  
    public void run() {  
        System.out.println(Thread.currentThread().getName() + " Priority: " +  
getPriority());  
    }  
  
    public static void main(String args[]) {  
        PriorityDemo t1 = new PriorityDemo();  
        PriorityDemo t2 = new PriorityDemo();  
  
        t1.setPriority(Thread.MIN_PRIORITY);  
        t2.setPriority(Thread.MAX_PRIORITY);  
  
        t1.start();  
        t2.start();  
    }  
}
```

4.6 Fundamentals of Exception Handling

An **Exception** is an unwanted event that interrupts normal program execution flow. Exception handling provides a mechanism to trap runtime errors and handle them gracefully without crashing the application.

4.7 Exception Handling Keywords

Keyword	Operational Purpose
<code>try</code>	Encloses code blocks that might throw runtime exceptions.
<code>catch</code>	Encloses exception handler code to process specific thrown exceptions.
<code>finally</code>	Encloses cleanup code guaranteed to execute regardless of whether an exception occurs.
<code>throw</code>	Explicitly throws a specific exception instance from code.
<code>throws</code>	Declares exceptions in a method signature that may be passed up the call stack.

```
class ExceptionHandlingDemo {  
    public static void main(String args[]) {  
        int a = 10, b = 0, result;  
  
        try {  
            System.out.println("Executing Division Operation...");  
            result = a / b; // Throws ArithmeticException  
            System.out.println("Result = " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Error Caught: Division by zero is undefined!");  
        } finally {  
            System.out.println("Finally block executed. Resources closed successfully.");  
        }  
    }  
}
```

4.8 Best Practices & Production Standards in Exception Handling

- Catch specific exceptions rather than relying on generic `Exception` handlers.
- Always cleanup system resources (files, sockets, database connections) inside `finally` blocks or try-with-resources.
- Avoid silent failures (empty catch blocks); log errors clearly to standard output or error logs.